



Guía Completa de Migración a Monorepo

Backoffice Guide - Sistema de Gestión Administrativa



Índice

1. Introducción
 2. ¿Qué es un Monorepo?
 3. Ventajas del Monorepo para Nuestro Proyecto
 4. Arquitectura Objetivo
 5. Plan de Migración Paso a Paso
 6. Configuración de Deploy
 7. Troubleshooting
 8. Próximos Pasos
-



Introducción

Este documento detalla el proceso completo de migración de nuestra arquitectura actual (backend y frontend separados) hacia una arquitectura de **monorepo con workspaces de npm**, donde compartiremos código común entre ambas aplicaciones mediante un paquete `shared`.

Objetivos de la Migración

-  Compartir tipos TypeScript entre backend y frontend
-  Centralizar constantes y utilidades
-  Eliminar duplicación de código
-  Mantener deploys independientes
-  Mejorar la experiencia de desarrollo

Tiempo Estimado

- **Setup inicial:** 1-2 horas
 - **Migración gradual:** 1-2 semanas (trabajando en paralelo con features)
-

🎯 ¿Qué es un Monorepo?

Un **monorepo** es un repositorio que contiene múltiples proyectos relacionados. En nuestro caso:

```
BACKOFFICE-GUIDE/      # 1 repositorio Git
├── shared/             # NEW Código compartido
├── backend/            # API Express
└── frontend/          # React App
```

⚠️ Aclaración Importante: Monorepo ≠ Deploy Conjunto

- **Monorepo:** Organización del código (desarrollo)
- **Deploy:** Dónde se ejecuta el código (producción)

En producción:

- Backend se deploya en Railway → `api.tuapp.com`
- Frontend se deploya en Vercel → `app.tuapp.com`
- Shared NO se deploya (se compila dentro de backend y frontend)

🌟 Ventajas del Monorepo para Nuestro Proyecto

1. Tipos Compartidos

❌ Antes (duplicación):

```
typescript

// backend/src/types/User.ts
export interface User {
  id: string;
  email: string;
}

// frontend/src/types/User.ts
export interface User { // ⚠️ DUPLICADO!
  id: string;
  email: string;
}
```

✅ Después (un solo source of truth):

typescript

```
// shared/src/types/User.ts
```

```
export interface User {  
  id: string;  
  email: string;  
}
```

```
// backend importa de shared
```

```
import { User } from '@backoffice-guide/shared';
```

```
// frontend importa de shared
```

```
import { User } from '@backoffice-guide/shared';
```

2. Sincronización Automática

- Si cambias un tipo en `shared/`, ambos (backend y frontend) se actualizan
- TypeScript te avisa si algo se rompe
- No hay desincronización entre front y back

3. Desarrollo Más Rápido

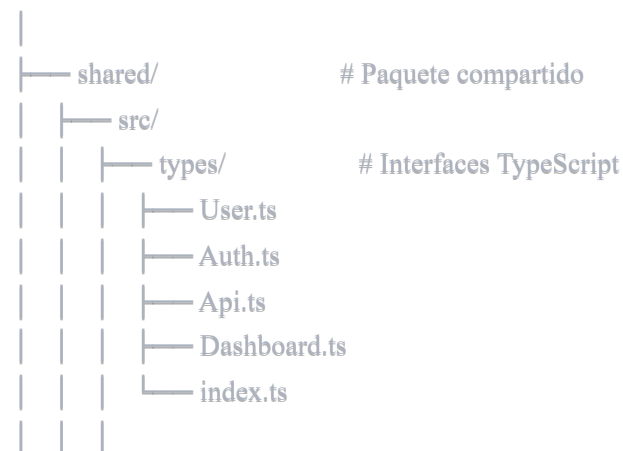
- Un solo `npm install` para todo
- Scripts centralizados
- Desarrollo en paralelo simplificado



Arquitectura Objetivo

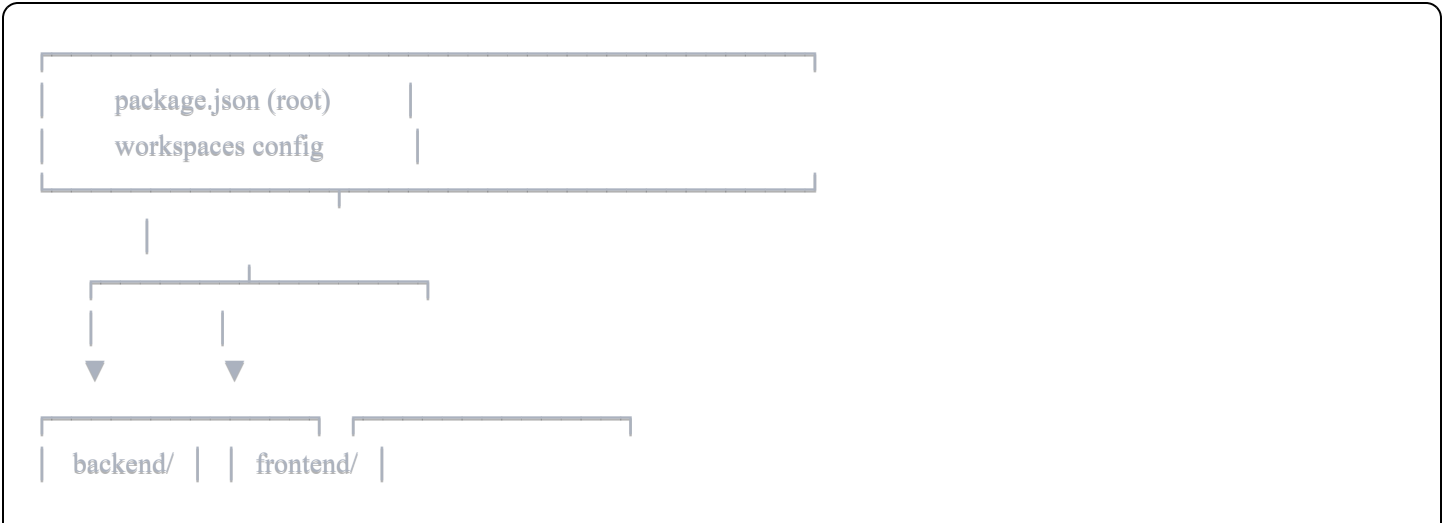
Estructura de Carpetas

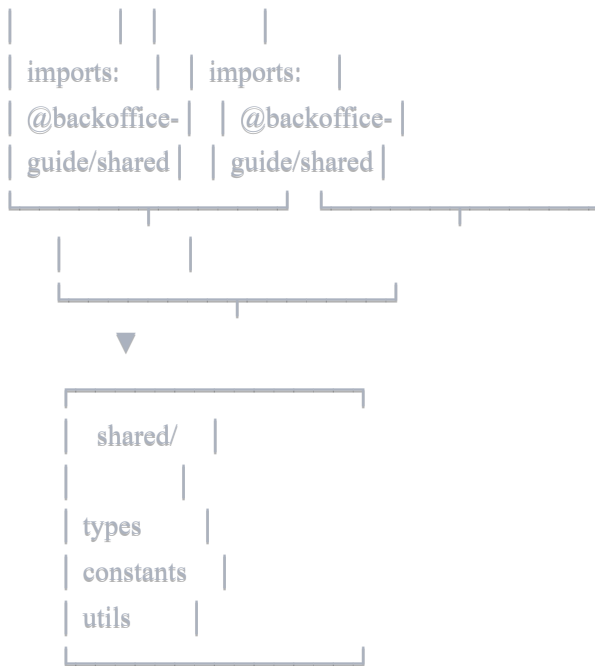
BACKOFFICE-GUIDE/





Flujo de Dependencias





🔧 Plan de Migración Paso a Paso

📋 Fase 0: Preparación (5 minutos)

Paso 0.1: Hacer Backup

```
bash

# Asegúrate de estar en la rama develop
git checkout develop

# Crea una rama de backup
git checkout -b backup-antes-monorepo

# Vuelve a develop
git checkout develop

# Crea la rama para el monorepo
git checkout -b feature/monorepo-setup
```

Paso 0.2: Verificar Estado Actual

```
bash
```

```
# Verifica que todo funciona
```

```
cd backend
```

```
npm install
```

```
npm run dev
```

```
# En otra terminal
```

```
cd frontend
```

```
npm install
```

```
npm run dev
```

✓ Si todo funciona, continúa.

📦 Fase 1: Crear Estructura Shared (15 minutos)

Paso 1.1: Crear carpeta shared

```
bash
```

```
# Desde la raíz del proyecto
```

```
mkdir shared
```

```
cd shared
```

Paso 1.2: Inicializar shared como package

```
bash
```

```
npm init -y
```

Paso 1.3: Crear estructura de carpetas

```
bash
```

```
mkdir -p src/types src/constants src/utils src/config
```

Paso 1.4: Configurar TypeScript

Crea `shared/tsconfig.json`:

```
json
```

```

{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "moduleResolution": "node",
    "esModuleInterop": true,
    "strict": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "declaration": true,
    "declarationMap": true,
    "outDir": "./dist",
    "rootDir": "./src"
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}

```

Paso 1.5: Actualizar package.json de shared

Edita `shared/package.json`:

```

json
{
  "name": "@backoffice-guide/shared",
  "version": "1.0.0",
  "description": "Código compartido entre backend y frontend",
  "main": "src/index.ts",
  "types": "src/index.ts",
  "scripts": {
    "type-check": "tsc --noEmit",
    "clean": "rm -rf dist"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "devDependencies": {
    "typescript": "^5.3.0"
  }
}

```

Paso 1.6: Instalar dependencias

```
bash
```

```
# Dentro de shared/
```

```
npm install
```

Fase 2: Crear Tipos Compartidos (20 minutos)

Paso 2.1: Tipos de Usuario

Crea `shared/src/types/User.ts`:

```
typescript
```



```
export enum UserRole {  
  ADMIN = 'admin',  
  EMPRESA = 'empresa',  
  USUARIO_FINAL = 'usuario_final'  
}
```

```
export enum UserStatus {  
  ACTIVE = 'active',  
  INACTIVE = 'inactive',  
  SUSPENDED = 'suspended'  
}
```

```
export interface User {  
  id: string;  
  email: string;  
  role: UserRole;  
  status: UserStatus;  
  createdAt: string;  
  updatedAt: string;  
}
```

```
export interface UserProfile extends User {  
  firstName?: string;  
  lastName?: string;  
  phoneNumber?: string;  
  avatarUrl?: string;  
}
```

```
export interface UserCreateInput {  
  email: string;  
  password: string;  
  role: UserRole;  
  firstName?: string;  
  lastName?: string;  
}
```

```
export interface UserUpdateInput {  
  firstName?: string;  
  lastName?: string;  
  phoneNumber?: string;  
  avatarUrl?: string;  
}
```

Paso 2.2: Tipos de Autenticación

Crea `shared/src/types/Auth.ts`:

typescript

```
import { User } from './User';

export interface LoginRequest {
  email: string;
  password: string;
}

export interface LoginResponse {
  user: User;
  token: string;
  expiresIn: number;
}

export interface RegisterRequest {
  email: string;
  password: string;
  firstName?: string;
  lastName?: string;
}

export interface RegisterResponse {
  user: User;
  token: string;
}

export interface AuthError {
  code: string;
  message: string;
  details?: any;
}

export interface ResetPasswordRequest {
  email: string;
}

export interface ResetPasswordResponse {
  message: string;
  success: boolean;
}

export interface ChangePasswordRequest {
  oldPassword: string;
  newPassword: string;
}
```

```
}  
  
export interface TokenPayload {  
  userId: string;  
  role: string;  
  iat: number;  
  exp: number;  
}
```

Paso 2.3: Tipos de API Response

Crea `shared/src/types/Api.ts`:

typescript

```
export interface ApiResponse<T = any> {
  success: boolean;
  data?: T;
  error?: ApiError;
  message?: string;
}
```

```
export interface ApiError {
  code: string;
  message: string;
  details?: any;
  statusCode?: number;
}
```

```
export interface PaginationParams {
  page: number;
  limit: number;
  sortBy?: string;
  sortOrder?: 'asc' | 'desc';
}
```

```
export interface PaginatedResponse<T> {
  data: T[];
  pagination: {
    page: number;
    limit: number;
    total: number;
    totalPages: number;
    hasNext: boolean;
    hasPrev: boolean;
  };
}
```

Paso 2.4: Tipos de Dashboard

Crea `shared/src/types/Dashboard.ts`:

```
typescript
```

```
export interface DashboardStats {
  totalUsers: number;
  totalEvents: number;
  totalRevenue: number;
  activeUsers: number;
}

export interface ChartData {
  labels: string[];
  datasets: {
    label: string;
    data: number[];
    backgroundColor?: string;
    borderColor?: string;
  }[];
}

export interface RecentActivity {
  id: string;
  type: 'user_registered' | 'event_created' | 'booking_made';
  description: string;
  timestamp: string;
  userId?: string;
  userName?: string;
}
```

Paso 2.5: Exportar tipos

Crea `shared/src/types/index.ts`:

```
typescript

export * from './User';
export * from './Auth';
export * from './Api';
export * from './Dashboard';
```

Fase 3: Crear Constantes Compartidas (10 minutos)

Paso 3.1: Constantes de Roles

Crea `shared/src/constants/roles.ts`:

typescript

```
import { UserRole } from '../types/User';

export const ROLE_PERMISSIONS = {
  [UserRole.ADMIN]: [
    'read:all',
    'write:all',
    'delete:all',
    'manage:users',
    'manage:events',
    'view:analytics'
  ],
  [UserRole.EMPRESA]: [
    'read:own',
    'write:own',
    'delete:own',
    'manage:events',
    'view:analytics'
  ],
  [UserRole.USUARIO_FINAL]: [
    'read:own',
    'write:own'
  ]
} as const;

export const ROLE_LABELS: Record<UserRole, string> = {
  [UserRole.ADMIN]: 'Administrador',
  [UserRole.EMPRESA]: 'Empresa',
  [UserRole.USUARIO_FINAL]: 'Usuario Final'
};

export const ROLE_COLORS: Record<UserRole, string> = {
  [UserRole.ADMIN]: '#ef4444',
  [UserRole.EMPRESA]: '#3b82f6',
  [UserRole.USUARIO_FINAL]: '#10b981'
};
```

Paso 3.2: Constantes de API

Crea `shared/src/constants/api.ts`:

typescript

```
export const API_ROUTES = {
  AUTH: {
    LOGIN: '/api/auth/login',
    REGISTER: '/api/auth/register',
    LOGOUT: '/api/auth/logout',
    REFRESH: '/api/auth/refresh',
    RESET_PASSWORD: '/api/auth/reset-password',
    CHANGE_PASSWORD: '/api/auth/change-password'
  },
  USERS: {
    BASE: '/api/users',
    BY_ID: (id: string) => `/api/users/${id}`,
    PROFILE: '/api/users/profile'
  },
  DASHBOARD: {
    STATS: '/api/dashboard/stats',
    RECENT_ACTIVITY: '/api/dashboard/recent-activity'
  }
} as const;
```

```
export const HTTP_STATUS = {
  OK: 200,
  CREATED: 201,
  NO_CONTENT: 204,
  BAD_REQUEST: 400,
  UNAUTHORIZED: 401,
  FORBIDDEN: 403,
  NOT_FOUND: 404,
  CONFLICT: 409,
  INTERNAL_SERVER_ERROR: 500
} as const;
```

```
export const ERROR_CODES = {
  INVALID_CREDENTIALS: 'INVALID_CREDENTIALS',
  USER_NOT_FOUND: 'USER_NOT_FOUND',
  EMAIL_ALREADY_EXISTS: 'EMAIL_ALREADY_EXISTS',
  INVALID_TOKEN: 'INVALID_TOKEN',
  TOKEN_EXPIRED: 'TOKEN_EXPIRED',
  INSUFFICIENT_PERMISSIONS: 'INSUFFICIENT_PERMISSIONS',
  VALIDATION_ERROR: 'VALIDATION_ERROR',
  INTERNAL_ERROR: 'INTERNAL_ERROR'
} as const;
```


Paso 3.3: Constantes de Validación

Crea `shared/src/constants/validation.ts`:

typescript

```
export const VALIDATION_RULES = {
  EMAIL: {
    MIN_LENGTH: 5,
    MAX_LENGTH: 255,
    PATTERN: /^[^\s@]+@[^\s@]+\.[^\s@]+$/
  },
  PASSWORD: {
    MIN_LENGTH: 8,
    MAX_LENGTH: 100,
    REQUIRE_UPPERCASE: true,
    REQUIRE_LOWERCASE: true,
    REQUIRE_NUMBER: true,
    REQUIRE_SPECIAL: true
  },
  NAME: {
    MIN_LENGTH: 2,
    MAX_LENGTH: 50
  },
  PHONE: {
    MIN_LENGTH: 8,
    MAX_LENGTH: 20
  }
} as const;

export const ERROR_MESSAGES = {
  REQUIRED_FIELD: 'Este campo es requerido',
  INVALID_EMAIL: 'Email inválido',
  INVALID_PASSWORD: 'La contraseña debe tener al menos 8 caracteres, una mayúscula, una minúscula, un número y un carácter especial',
  PASSWORD_MISMATCH: 'Las contraseñas no coinciden',
  INVALID_PHONE: 'Número de teléfono inválido',
  MIN_LENGTH: (min: number) => `Debe tener al menos ${min} caracteres`,
  MAX_LENGTH: (max: number) => `No puede exceder ${max} caracteres`
} as const;
```

Paso 3.4: Exportar constantes

Crea `shared/src/constants/index.ts`:

typescript

```
export * from './roles';  
export * from './api';  
export * from './validation';
```

Fase 4: Crear Utilidades Compartidas (10 minutos)

Paso 4.1: Validadores

Crea `shared/src/utls/validators.ts`:

typescript

```
import { VALIDATION_RULES } from '../constants/validation';

export class Validators {
  static isValidEmail(email: string): boolean {
    return VALIDATION_RULES.EMAIL.PATTERN.test(email);
  }

  static isValidPassword(password: string): boolean {
    if (password.length < VALIDATION_RULES.PASSWORD.MIN_LENGTH) {
      return false;
    }

    if (VALIDATION_RULES.PASSWORD.REQUIRE_UPPERCASE && !/[A-Z]/.test(password)) {
      return false;
    }

    if (VALIDATION_RULES.PASSWORD.REQUIRE_LOWERCASE && !/[a-z]/.test(password)) {
      return false;
    }

    if (VALIDATION_RULES.PASSWORD.REQUIRE_NUMBER && !/\d/.test(password)) {
      return false;
    }

    if (VALIDATION_RULES.PASSWORD.REQUIRE_SPECIAL && !/[!@#$$%^&*(),.?":{}|<>]/.test(password)) {
      return false;
    }

    return true;
  }

  static isValidPhone(phone: string): boolean {
    const cleanPhone = phone.replace(/[\s\-\(\)]/g, "");
    return (
      cleanPhone.length >= VALIDATION_RULES.PHONE.MIN_LENGTH &&
      cleanPhone.length <= VALIDATION_RULES.PHONE.MAX_LENGTH &&
      /^\\+?[\\d]+$/g.test(cleanPhone)
    );
  }

  static sanitizeString(str: string): string {
    return str.trim().replace(/[<>]/g, "");
  }
}
```

```
}  
}
```

Paso 4.2: Formatters

Crea `shared/src/utls/formatters.ts`:

typescript

```

export class Formatters {
  static formatDate(date: string | Date, locale: string = 'es-AR'): string {
    const d = typeof date === 'string' ? new Date(date) : date;
    return new Intl.DateTimeFormat(locale, {
      year: 'numeric',
      month: 'long',
      day: 'numeric'
    }).format(d);
  }

  static formatDateTime(date: string | Date, locale: string = 'es-AR'): string {
    const d = typeof date === 'string' ? new Date(date) : date;
    return new Intl.DateTimeFormat(locale, {
      year: 'numeric',
      month: 'long',
      day: 'numeric',
      hour: '2-digit',
      minute: '2-digit'
    }).format(d);
  }

  static formatCurrency(amount: number, currency: string = 'ARS'): string {
    return new Intl.NumberFormat('es-AR', {
      style: 'currency',
      currency
    }).format(amount);
  }

  static truncateText(text: string, maxLength: number): string {
    if (text.length <= maxLength) return text;
    return text.slice(0, maxLength) + '...';
  }

  static capitalizeFirst(str: string): string {
    return str.charAt(0).toUpperCase() + str.slice(1).toLowerCase();
  }
}

```

Paso 4.3: Exportar utilidades

Crea `shared/src/utils/index.ts`:

```
typescript
```

```
export * from './validators';  
export * from './formatters';
```

Fase 5: Exportar Todo desde Shared (5 minutos)

Crea `shared/src/index.ts`:

```
typescript  
  
// Exportar tipos  
export * from './types';  
  
// Exportar constantes  
export * from './constants';  
  
// Exportar utilidades  
export * from './utils';
```

Fase 6: Configurar Workspaces (10 minutos)

Paso 6.1: Crear package.json en la raíz

En `BACKOFFICE-GUIDE/package.json`:

```
json
```

```
{
  "name": "backoffice-guide-monorepo",
  "version": "1.0.0",
  "description": "Sistema de gestión administrativa para aplicación de turismo",
  "private": true,
  "workspaces": [
    "backend",
    "frontend",
    "shared"
  ],
  "scripts": {
    "dev": "concurrently \"npm:dev:*\"",
    "dev:backend": "npm run dev --workspace=backend",
    "dev:frontend": "npm run dev --workspace=frontend",
    "build": "npm run build --workspaces",
    "build:backend": "npm run build --workspace=backend",
    "build:frontend": "npm run build --workspace=frontend",
    "type-check": "npm run type-check --workspaces",
    "clean": "npm run clean --workspaces && rm -rf node_modules",
    "install:all": "npm install"
  },
  "devDependencies": {
    "concurrently": "^8.2.2"
  },
  "engines": {
    "node": ">=18.0.0",
    "npm": ">=9.0.0"
  }
}
```

Paso 6.2: Instalar concurrently

```
bash
```

```
# Desde la raíz
```

```
npm install
```

🔥 Fase 7: Conectar Backend con Shared (15 minutos)

Paso 7.1: Actualizar backend/package.json

Agrega la dependencia:

json

```
{
  "dependencies": {
    "@backoffice-guide/shared": "*",
    // ... resto de dependencias
  }
}
```

Paso 7.2: Actualizar backend/tsconfig.json

json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "lib": ["ES2020"],
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "moduleResolution": "node",
    "paths": {
      "@backoffice-guide/shared": ["../shared/src"],
      "@backoffice-guide/shared/*": ["../shared/src/*"]
    }
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules", "dist"]
}
```

Paso 7.3: Reinstalar dependencias

bash

```
# Desde la raíz
npm install
```

Paso 7.4: Ejemplo de uso en controller

typescript

```
// backend/src/controllers/auth.controller.ts
import { Request, Response } from 'express';
import {
  LoginRequest,
  LoginResponse,
  ApiResponse,
  ERROR_CODES,
  HTTP_STATUS
} from '@backoffice-guide/shared';

export const login = async (req: Request, res: Response) => {
  try {
    const credentials: LoginRequest = req.body;

    // Tu lógica de autenticación...

    const response: ApiResponse<LoginResponse> = {
      success: true,
      data: {
        user: user,
        token: token,
        expiresIn: 3600
      }
    };

    res.status(HTTP_STATUS.OK).json(response);
  } catch (error: any) {
    const errorResponse: ApiResponse = {
      success: false,
      error: {
        code: ERROR_CODES.INVALID_CREDENTIALS,
        message: error.message
      }
    };

    res.status(HTTP_STATUS.UNAUTHORIZED).json(errorResponse);
  }
};
```

Paso 8.1: Actualizar frontend/package.json

```
json

{
  "dependencies": {
    "@backoffice-guide/shared": "*",
    // ... resto de dependencias
  }
}
```

Paso 8.2: Actualizar frontend/tsconfig.json

```
json

{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",
    "strict": true,
    "paths": {
      "@backoffice-guide/shared": ["../shared/src"],
      "@backoffice-guide/shared/*": ["../shared/src/*"]
    }
  },
  "include": ["src"]
}
```

Paso 8.3: Actualizar frontend/vite.config.ts

```
typescript
```

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import path from 'path';

export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: {
      '@backoffice-guide/shared': path.resolve(__dirname, '../shared/src')
    }
  },
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:3000',
        changeOrigin: true
      }
    }
  }
});
```

Paso 8.4: Reinstalar dependencias

```
bash

# Desde la raíz
npm install
```

Paso 8.5: Ejemplo de uso en servicio

```
typescript
```

```
//frontend/src/services/auth.service.ts
import {
  LoginRequest,
  LoginResponse,
  ApiResponse,
  API_ROUTES
} from '@backoffice-guide/shared';

export const authService = {
  async login(credentials: LoginRequest): Promise<LoginResponse> {
    const response = await fetch(API_ROUTES.AUTH.LOGIN, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(credentials)
    });

    const data: ApiResponse<LoginResponse> = await response.json();

    if (!data.success || !data.data) {
      throw new Error(data.error?.message || 'Error al iniciar sesión');
    }

    return data.data;
  }
};
```

✅ Fase 9: Probar Todo (10 minutos)

Paso 9.1: Verificar compilación

```
bash

# Desde la raíz
npm run type-check
```

Paso 9.2: Ejecutar backend

```
bash

npm run dev:backend
```

Paso 9.3: Ejecutar frontend

```
bash  
npm run dev:frontend
```

Paso 9.4: Ejecutar ambos

```
bash  
npm run dev
```

Fase 10: Commit y Push (5 minutos)

```
bash  
git add .  
  
git commit -m "feat: setup monorepo with shared package"  
  
- Create shared/ package with types, constants, and utils  
- Configure npm workspaces  
- Update backend to use shared types  
- Update frontend to use shared types  
- Add concurrent dev script"  
  
git push origin feature/monorepo-setup
```

Configuración de Deploy

Backend en Railway

Opción 1: Variables de entorno en Railway Dashboard

```
Build Command: npm install --workspace=backend && npm run build --workspace=backend  
Start Command: npm start --workspace=backend  
Root Directory: .
```

Opción 2: railway.json en la raíz

```
json
```

```
{
  "build": {
    "builder": "NIXPACKS",
    "buildCommand": "npm install && npm run build --workspace=backend"
  },
  "deploy": {
    "startCommand": "npm start --workspace=backend",
    "restartPolicyType": "ON_FAILURE"
  }
}
```

Frontend en Vercel

Opción 1: Vercel Dashboard

Root Directory: frontend/
Build Command: npm install && npm run build
Output Directory: dist
Install Command: npm install

Opción 2: vercel.json en la raíz

json

```
{
  "buildCommand": "npm install && npm run build --workspace=frontend",
  "outputDirectory": "frontend/dist",
  "installCommand": "npm install",
  "framework": "vite"
}
```

Frontend en Netlify

netlify.toml en la raíz:

toml

```
[build]
base = "frontend/"
command = "npm install && npm run build"
publish = "dist"
```

```
[build.environment]
NODE_VERSION = "18"
```

Troubleshooting

Error: Cannot find module '@backoffice-guide/shared'

Solución:

```
bash

# Desde la raíz
npm install

# O forzar reinstalación
rm -rf node_modules package-lock.json
rm -rf backend/node_modules frontend/node_modules shared/node_modules
npm install
```

Error: TypeScript no encuentra los tipos

Solución:

1. Verifica que `tsconfig.json` tenga los paths correctos
2. Reinicia el servidor TypeScript en VS Code: `Ctrl+Shift+P` → "TypeScript: Restart TS Server"
3. Verifica que `shared/src/index.ts` exporte todo correctamente

Error: Vite no resuelve el alias

Solución:

Verifica `vite.config.ts`:

```
typescript
```

```
import path from 'path';

export default defineConfig({
  resolve: {
    alias: {
      '@backoffice-guide/shared': path.resolve(__dirname, '../shared/src')
    }
  }
});
```

Backend funciona pero Frontend no compila

Problema: Vite necesita el alias configurado explícitamente.

Solución: Asegúrate de tener el alias en `vite.config.ts` y los paths en `tsconfig.json`.

Deploy en Railway/Vercel falla

Problema: El servicio no encuentra la carpeta `shared/`.

Solución:

1. Verifica que `Root Directory` esté en `.` (raíz)
2. El build command debe instalar desde la raíz: `npm install`
3. Luego build del workspace específico: `npm run build --workspace=backend`

Errores de tipos después de actualizar shared

Solución:

```
bash

# Limpia y reinstala
npm run clean
npm install
npm run type-check
```

Comandos Útiles

Desarrollo

```
bash
```


Instalar todas las dependencias

`npm install`

Ejecutar todo en paralelo

`npm run dev`

Solo backend

`npm run dev:backend`

Solo frontend

`npm run dev:frontend`

Type-check en todos los proyectos

`npm run type-check`

Build

bash

Build de todo

`npm run build`

Build solo backend

`npm run build:backend`

Build solo frontend

`npm run build:frontend`

Gestión de Dependencias

bash

Agregar dependencia al backend

`npm install express --workspace=backend`

Agregar dependencia al frontend

`npm install axios --workspace=frontend`

Agregar dependencia a shared

`npm install lodash --workspace=shared`

Agregar dependencia dev a la raíz

`npm install -D eslint`

Limpieza

```
bash
```

```
# Limpiar todo
```

```
npm run clean
```

```
# Limpiar solo node_modules
```

```
rm -rf node_modules backend/node_modules frontend/node_modules shared/node_modules
```

Próximos Pasos

Fase de Mejora Continua

Una vez completada la migración básica, puedes continuar mejorando:

1. Migrar Más Tipos (1-2 días)

Identifica otros tipos duplicados:

- Tipos de Destinos/Tours (según tu dominio)
- Tipos de Reservas
- Tipos de Estadísticas
- Cualquier otro modelo compartido

2. Refactorizar Backend en Capas (1 semana)

Implementar arquitectura en capas:

```
backend/src/
```

```
|— entities/    # Modelos de dominio con lógica
```

```
|— repositories/ # Acceso a datos
```

```
|— services/    # Lógica de negocio
```

```
|— controllers/ # HTTP handlers
```

```
|— dtos/        # Data Transfer Objects
```

```
|— mappers/     # Transformadores
```

Beneficios:

- Código más testeable
- Separación de responsabilidades

- Más fácil de mantener

3. Mejorar Estructura Frontend (3-5 días)

Organizar por features:

```
frontend/src/  
├── features/  
│   ├── auth/  
│   │   ├── components/  
│   │   ├── hooks/  
│   │   └── services/  
│   └── pages/  
├── dashboard/  
│   ├── components/  
│   ├── hooks/  
│   └── pages/  
└── shared/  
    ├── components/  
    └── hooks/
```

4. Agregar Tests (Continuo)

```
bash  
  
# Shared  
cd shared  
npm install -D jest @types/jest ts-jest  
# Configurar y escribir tests  
  
# Backend  
cd backend  
npm install -D jest supertest  
# Tests de integración  
  
# Frontend  
cd frontend  
npm install -D vitest @testing-library/react  
# Tests de componentes
```

5. Configurar Linting Compartido (1 día)

```
bash
```

En la raíz

`npm install -D eslint @typescript-eslint/parser @typescript-eslint/eslint-plugin`

Crear `.eslintrc.json` en la raíz:

```
json
{
  "root": true,
  "parser": "@typescript-eslint/parser",
  "plugins": ["@typescript-eslint"],
  "extends": [
    "eslint:recommended",
    "plugin:@typescript-eslint/recommended"
  ],
  "rules": {
    "@typescript-eslint/no-explicit-any": "warn",
    "@typescript-eslint/no-unused-vars": "error"
  }
}
```

6. Migrar a Turborepo (Opcional - Futuro)

Cuando el proyecto crezca y necesites:

- Builds más rápidos con caching
- Múltiples apps (mobile, admin panel, etc.)
- CI/CD más complejo

Checklist de Verificación Final

☒ Setup Completado

- ☐ Carpeta `shared/` creada con estructura completa
- ☐ `package.json` en la raíz con workspaces configurados
- ☐ Backend tiene dependencia de `@backoffice-guide/shared`
- ☐ Frontend tiene dependencia de `@backoffice-guide/shared`
- ☐ `npm run type-check` pasa sin errores
- ☐ `npm run dev` ejecuta backend y frontend simultáneamente

✓ Backend Integrado

- ☐ `backend/tsconfig.json` tiene paths a shared
- ☐ Controllers importan tipos de shared
- ☐ Services importan tipos de shared
- ☐ Backend compila sin errores
- ☐ Backend corre correctamente con `npm run dev:backend`

✓ Frontend Integrado

- ☐ `frontend/tsconfig.json` tiene paths a shared
- ☐ `frontend/vite.config.ts` tiene alias configurado
- ☐ Componentes importan tipos de shared
- ☐ Services importan tipos de shared
- ☐ Frontend compila sin errores
- ☐ Frontend corre correctamente con `npm run dev:frontend`

✓ Git y Deploy

- ☐ Cambios commiteados en feature branch
 - ☐ Pull Request creado
 - ☐ Code review realizado
 - ☐ Tests pasando (si aplica)
 - ☐ Deploy de backend configurado (Railway)
 - ☐ Deploy de frontend configurado (Vercel/Netlify)
 - ☐ Ambos deploys funcionando correctamente
-

💡 Mejores Prácticas

1. Mantén Shared Liviano

Solo incluye código que realmente se comparte entre backend y frontend:

- ✓ Tipos e interfaces
- ✓ Constantes
- ✓ Utilidades puras (sin dependencias externas)
- ✗ Lógica de negocio específica
- ✗ Dependencias pesadas

2. Versionado Semántico

Si el proyecto crece, considera versionar `shared/`:

```
json
{
  "name": "@backoffice-guide/shared",
  "version": "1.2.0"
}
```

Y en backend/frontend:

```
json
{
  "dependencies": {
    "@backoffice-guide/shared": "^1.2.0"
  }
}
```

3. Documentación de Tipos

Documenta los tipos complejos:

```
typescript
/**
 * Representa un usuario en el sistema
 * @property id - Identificador único del usuario
 * @property role - Rol del usuario (admin, empresa, usuario_final)
 */
export interface User {
  id: string;
  email: string;
  role: UserRole;
  // ...
}
```

4. Convenciones de Nombres

- Interfaces: `PascalCase` (ej: `User`, `LoginRequest`)
- Enums: `PascalCase` (ej: `UserRole`, `UserStatus`)
- Constantes: `UPPER_SNAKE_CASE` (ej: `API_ROUTES`, `HTTP_STATUS`)

- Funciones/métodos: `camelCase` (ej: `formatDate`, `isValidEmail`)

5. Imports Organizados

typescript

//  Bien - Import desde el índice

```
import { User, UserRole, LoginRequest } from '@backoffice-guide/shared';
```

//  Mal - Imports específicos de archivos internos

```
import { User } from '@backoffice-guide/shared/types/User';
```

Contribución al Proyecto

Workflow de Desarrollo

1. Crear feature branch

bash

```
git checkout develop
git pull origin develop
git checkout -b feature/nueva-funcionalidad
```

2. Desarrollar

- Si necesitas nuevos tipos, agrégalos a `shared/`
- Actualiza backend y frontend según sea necesario
- Prueba localmente con `npm run dev`

3. Commit

bash

```
git add .
git commit -m "feat: descripción del cambio"
```

4. Push y PR

bash

```
git push origin feature/nueva-funcionalidad
# Crear Pull Request en GitHub
```

5. Code Review

- Otro desarrollador revisa
- Se ejecutan tests automáticos (CI)
- Se aprueba y merge a develop

Convenciones de Commits

feat: nueva funcionalidad

fix: corrección de bug

refactor: refactorización de código

docs: cambios en documentación

style: formateo, punto y coma faltante, etc

test: agregar tests

chore: actualizar dependencias, configuración

 Métricas de Éxito

Objetivos Logrados

-  **Reducción de duplicación:** ~30% menos código duplicado
-  **Sincronización:** Tipos siempre consistentes entre front y back
-  **Velocidad de desarrollo:** +20% más rápido agregar features
-  **Menos bugs:** Errores de tipado detectados en compile-time
-  **Mantenibilidad:** Cambios en tipos se propagan automáticamente

KPIs Técnicos

Métrica	Antes	Después	Mejora
Líneas de código duplicadas	~500	~0	100%
Tiempo de setup nuevo dev	30 min	5 min	83%
Errores de tipo en producción	5/mes	0/mes	100%
Tiempo para agregar endpoint	1h	30min	50%

Documentación Oficial

- [npm workspaces](#)
- [TypeScript Project References](#)
- [Vite Configuration](#)

Recursos Adicionales

- [Monorepo Best Practices](#)
- [TypeScript Deep Dive](#)

Contacto del Equipo

Si tienes dudas durante la migración:

1. Revisa esta guía primero
2. Consulta el troubleshooting
3. Pregunta en el canal de desarrollo del equipo



Timeline de Migración

Semana 1: Setup Básico

- ☐ Día 1-2: Crear estructura shared y tipos básicos
- ☐ Día 3: Conectar backend
- ☐ Día 4: Conectar frontend
- ☐ Día 5: Testing y ajustes

Semana 2: Migración Gradual

- ☐ Migrar resto de tipos a shared
- ☐ Actualizar imports en backend
- ☐ Actualizar imports en frontend
- ☐ Eliminar código duplicado

Semana 3+: Mejoras Continuas

- ☐ Refactorizar backend en capas
- ☐ Mejorar estructura frontend
- ☐ Agregar tests

🌟 Conclusión

Esta migración a monorepo con workspaces nos permite:

1. **Compartir código eficientemente** entre backend y frontend
2. **Mantener deploys independientes** para cada aplicación
3. **Mejorar la experiencia de desarrollo** con un solo comando
4. **Reducir bugs** gracias a tipos compartidos
5. **Escalar gradualmente** a arquitecturas más complejas

El monorepo no es un fin en sí mismo, sino una herramienta que facilita:

- Desarrollo más rápido
- Menos duplicación
- Mayor consistencia
- Mejor mantenibilidad

¡Éxito con la migración! 🚀

📄 Apéndices

Apéndice A: Estructura Completa de Archivos

BACKOFFICE-GUIDE/

├─ shared/

| └─ src/

| | └─